

MULTI-SOURCE CANDIDATE DATA TRANSFORMER

Technical Architecture Design & Step-by-Step Project Execution Blueprint

1. Executive Problem Framing & Principles

The downstream technical consumers require a single, clean, canonical profile per candidate. The engineering execution path must reconcile structured (CSV, ATS JSON) and unstructured (GitHub, LinkedIn, Resume PDFs) inputs while managing formatting drift, structural gaps, and data contradictions.

Core Objective Function

Maximize data synthesis fidelity while strictly adhering to the premise: **"Wrong-but-confident is worse than honestly-empty."** Hallucinations or erroneous joins silently degrade hiring analytics; structural gaps are infinitely preferable to incorrect data claims.

2. Complete Pipeline Breakdown (Sequential Processing Stages)

The framework operates via a unidirectional, decoupled pipeline to maintain strict isolation between raw ingestion data and consumer projections.

- Detect & Ingest:** Stream individual source inputs via designated adapters. For unstructured files (PDF, TXT), text layers are safely extracted. Each record is marked with an immutable system-level `source_id` and ingestion timestamp.
- Extract & Field-Map:** Standardize incoming JSON fields or CSV columns to the internal engine fields. Unstructured streams pass through deterministic regex extractors or structured entity recognizers to pull names, phone variants, email coordinates, and specific block structures (e.g., career or education items).
- Normalize:** Cast all successfully mapped raw attributes into unified schemas. Unmatched text forms are cleaned, stripped of padding, and prepared for identity indexing.
- Identity Matching (Resolution):** Apply a highly strict identity heuristic. If two inputs contain a matching normalized email or a matching E.164 primary phone number, they are dynamically grouped into a unified candidate bucket. Cross-matching on names alone is explicitly barred to avoid collisions.
- Merge & Conflict Reconciliation:** Evaluate colliding field attributes across disparate records using a deterministic weight matrix. Provenance chains are generated concurrently.
- Confidence Scoring:** Compute individual property and global entity confidence coefficients.
- Projection Layer (Runtime Configuration):** Intercept the internal canonical record and format it dynamic-on-demand according to runtime config specs (renaming paths, pruning fields, shifting structural shapes).
- Validate & Emit:** Validate the processed data structure against the runtime schema. Faulty configurations throw systematic errors, while successful entities are formatted cleanly as JSON.

3. Canonical Output Schema & Standardized Formats

The internal data matrix guarantees standard structures prior to rendering custom configurations:

Field Name	Internal Type / Shape	Normalization Rules & Target Standard
candidate_id	string	Deterministic UUIDv5 calculated using the lower-cased primary email.
full_name	string	Title-cased, whitespace collapsed, non-alphabetic noise characters stripped.
emails	string[]	Lower-cased, validated via standard regex, array sorted alphabetically to ensure determinism.
phones	string[]	Strict E.164 compliance via international prefixes (e.g., +14155552671). Invalid tokens dropped.
location	{city, region, country}	Country must map to strict ISO-3166 alpha-2. Structural mismatches fall back to regional text lookups.
links	object	Structured as {linkedin, github, portfolio, other[]}. Domain-sorted URL tokens.
years_experience	number null	Calculated by summing explicitly parsed continuous career durations, rounded to one decimal place.
skills	object[]	Array of maps: {name, confidence, sources[]}. Lower-cased, unified via synonym dictionary.
experience	object[]	List of {company, title, start, end, summary}. System dates standardized strictly to YYYY-MM .
provenance	object[]	Comprehensive audit trail tracking field mutations: {field, source, method} .
overall_confidence	number	Float between 0.00 and 1.00 representing internal data consistency.

4. Conflict Resolution & Confidence Calculations

A. Field Tiering Policy (Authority Matrix)

When disparate sources generate contradictory values for unique scalar attributes, the engine defaults to a deterministic tiering system instead of arbitrary overwrite patterns:

- **Tier 1 (Highest Confidence: 0.95):** ATS JSON Blobs / Verified Recruiter Exports (Highest accuracy for legal employment status, historical company names, titles).
- **Tier 2 (High Confidence: 0.85):** First-Party Sourced Resume Files (.docx / .pdf text blocks parsing professional timelines).
- **Tier 3 (Medium Confidence: 0.75):** Live API URLs (GitHub, LinkedIn profiles reflecting self-curated, dynamic assets).
- **Tier 4 (Lowest Confidence: 0.50):** Raw Recruiter Notes (.txt scribbles possessing high risk of typos or observational bias).

B. Algorithmic Confidence Formulation

The overall entity trust profile is calculated deterministically across the active attributes. Let w_i represent the reliability weight of the winning source for field i , and let C_{harmony} represent an internal consistency multiplier defined by the ratio of overlapping sources providing identical text strings.

$$\text{Overall Confidence} = (\sum_{i=1}^N (w_i) / N) \times C_{\text{harmony}}$$

Where $C_{\text{harmony}} = 1.00$ if overlapping files match exactly, and drops down to 0.80 if active sources state contradictory values for primary fields like `years_experience` or `current_company`.

5. Runtime Output Configuration & Projection Layer

The projection layer functions as an independent functional pipeline wrapper. It intercepts the immutable, fully processed internal record and processes mutations dynamically based on the incoming configuration object:

- **Pruning and Selection:** Iterates over the requested `fields` array. Properties omitted from the configuration block are dropped immediately from the memory map.
- **Path Resolution & Aliasing:** Supports explicit pointer configurations. For instance, evaluating a path directive mapping `primary_email` from `emails[0]` relies on standard internal pointer evaluation blocks to parse elements from nested arrays.
- **Missing Value Enforcement (`on_missing`):**
 - `"null"`: Standardizes missing field variables explicitly to a JSON null value.
 - `"omit"`: Uses JavaScript/Python memory deletions to prune keys out of final objects entirely.
 - `"error"`: Aborts operations immediately, raising a precise custom tracking exception.

6. Edge Cases & Defensive Engineering Guardrails

1. **Circular or Identity Loop Anomalies:** A scenario where Candidate A shares an email with Candidate B on a recruiter note, but a separate phone match links Candidate B to Candidate C.
Mitigation: Strict multi-key grouping isolation. Identity mapping will require unique, verifiable email matches as an absolute prerequisite. Secondary overlaps (like phone numbers) will trigger an isolation boundary flag instead of a forced merge.
2. **Corrupted Unstructured Text Extractor Feeds:** Resume files containing unparseable characters or empty image layers.
Mitigation: Fail-soft fallback bounds. Unparseable files must never crash the thread execution engine. The parser logs a localized system event, leaves unstructured entries empty, and shifts processing resources to the remaining structured inputs.
3. **Conflicting Array Metrics (Overlapping Timelines):** Employment items indicating the candidate held full-time engineering roles concurrently at two separate enterprises.
Mitigation: De-duplicate intervals using date boundary verification checks. Intersect overlapping ranges and flag potential data discrepancies within the profile's internal metadata tracking matrix.

7. Implementation Roadmap & Verification Milestones

To ensure verification at each stage, development is divided into four distinct test-driven phases:

Execution Phase	Core Deliverables & Focal Components	Verification Mechanism
Phase 1: Ingestion	Build individual file parsers for CSV rows and ATS JSON objects; integrate structural PDF string extractors.	Unit test suite using corrupted files to verify fail-soft resilience.
Phase 2: Core Engine	Develop phone E.164 normalization, date standardizer, identity-resolving maps, and deterministic merging loops.	Gold-profile snapshot comparison validating synthesized outputs against expectations.
Phase 3: Projection	Implement runtime path configuration interpreter, alias-mapping processors, and parameter filters.	Validation of multiple schema variants using schema compliance test fixtures.
Phase 4: Interface	Wrap system logic into a fast CLI tool; generate documentation, user setup instructions, and create a 2-minute demo clip.	End-to-end execution script utilizing live terminal pipeline parameters.